

☆ CCNA ☆

Revision Notes

*Lessons 1 – 10 · quick review notes for fast revision
(condensed from your Jeremy' IT Lab lesson notes)*



switches connect devices inside a LAN · routers connect LANs together

How to use these notes

*Read a lesson · cover the page · say the key facts out loud
Memorize the red + green boxes — they are what the exam loves*

What's inside

L1 · Network Devices	pg. 3
L2 · Interfaces & Cables	pg. 5
L3 · TCP/IP & OSI Models	pg. 7
L4 · Cisco IOS CLI	pg. 9
L5 · LAN Switching I	pg. 11
L6 · ARP, Ping & MAC Table	pg. 13
L7 · IPv4 Addressing	pg. 15
L8 · IPv4 2 + Router Config	pg. 17
L9 · Switch Interfaces	pg. 19
L10 · IPv4 Header	pg. 21
Cram Card · numbers & formulas	pg. 23

BEFORE THE EXAM

- Skim only the **highlighted** bits, the **tables**, and the green + red boxes.
- Rewrite the **cram card** (last page) once from memory — if you can, you're ready.
- Re-do the binary and usable-host drills until they take under 30 seconds each.

Lesson 1 — Network Devices ☆

A **network** = **nodes** (devices) connected so they can **share resources** (files, data, services). Even just **2 PCs on one cable** already counts as a network.

The 5 key devices

Device	What it does	Remember it as
Router	connects different networks (LAN · LAN, LAN · Internet)	few ports; the only way out to the Internet
Switch	connects end hosts inside one LAN	24+ ports; cannot join LANs or reach the Internet
Firewall	monitors / controls traffic using security rules	can sit inside, outside, or both around the router
Server	provides a service or resource	a role, not a machine type
Client	requests / uses a service	also just a role

Servers + clients together are called **end hosts / endpoints** — the devices that actually **use** the network; routers, switches and firewalls just move or police the traffic.

Client vs server = a ROLE, not hardware

- PC1 asks PC2 for **image.jpg** · PC1 = **client**, PC2 = **server**. Swap who asks, and the roles swap.
- Watching YouTube: your phone = client, YouTube's machine = server.
- AirDrop between two iPhones: the **sending** phone is the server, the **receiving** one is the client.

Two kinds of firewall

- **Network firewall** — hardware box guarding the whole network (Cisco ASA 5500-X, Firepower).
- **Host-based firewall** — software on each PC/phone guarding that one machine.
- **Best practice = use BOTH** (defense in depth). A **next-generation firewall (NGFW)** adds modern features like **IPS** (intrusion prevention).

How data travels (NY office · Tokyo office)



Switches deliver **within** a LAN; routers deliver **between** LANs. The reply traces the same path in reverse.

MUST REMEMBER

- **Switch** = inside one LAN, many ports, no Internet on its own
- **Router** = between LANs + the Internet, fewer ports
- **Firewall** = filters traffic in/out by rules (network + host-based)
- Client / server are **roles in one exchange** — one device can be both
- End hosts = PCs & servers; the rest is plumbing

Lesson 2 — Interfaces & Cables ☆

All **Layer 1 (physical)** stuff: connectors, cables, signals. **RJ-45** = the standard copper port/plug (8 pins).

Ethernet = a **family** of standards (**IEEE 802.3**), not one protocol; standards exist so every vendor's gear fits and talks together.

Bits vs bytes (do not mix these!)

- **1 byte = 8 bits** · uppercase **B** = byte, lowercase **b** = bit.
- Network speed is measured in **bits/sec** (a cable sends one bit at a time); storage is measured in **bytes**.
· So **1 GB** is **8x more** than **1 Gb**.
- **Kb · Mb · Gb · Tb** = each **x1,000**.

Copper Ethernet standards (twisted pair, RJ-45)

Name	Speed	Max length	Wire pairs used
10BASE-T	10 Mbps	100 m	2 pairs
100BASE-T	100 Mbps	100 m	2 pairs
1000BASE-T	1 Gbps	100 m	all 4, bidirectional
10GBASE-T	10 Gbps	100 m	all 4, bidirectional

Decoding names: number = speed · **BASE** = baseband signaling · **T** = **twisted pair** · **UTP** = Unshielded

Twisted Pair: 4 pairs / 8 wires, no shielding (twisting itself fights EMI). **All copper limits: 100 m.**

Which pins do what (10BASE-T / 100BASE-T)

Device	Pins 1 & 2	Pins 3 & 6
PC / Router / Firewall	Transmit (Tx)	Receive (Rx)
Switch (the odd one out)	Receive (Rx)	Transmit (Tx)

- One pair per direction · this is exactly why **full duplex** (send + receive at once) works.

Straight-through vs crossover

- **Straight-through** (pin 1 · pin 1): use between **opposite pin roles** **PC-SW** , **R-SW** .
- **Crossover** (1-3, 2-6 swapped): use between **same pin roles** **R-R** , **SW-SW** , **PC-PC** , **PC-R** .
- Memory hook: same type · **cross** over. Different type · straight through.
- **Auto MDI-X** (on all modern gear) auto-detects and adjusts Tx/Rx — **cable type no longer matters** .
Old gear without it still needs the correct cable.

Fiber-optic = for longer distances

- Sends data as **pulses of light through glass** · **immune to EMI** and no signal leakage (more secure) · but pricier (cables + **SFP** ports/transceivers).
- Uses **two strands** : one to transmit, one to receive (Tx on one end always feeds Rx on the other).
- Layers inside out: **core** (light travels here) · **cladding** (reflects light back in) · buffer · jacket.

	Multimode (MMF)	Single-mode (SMF)
Core	wider	narrower
Light paths	many ("modes")	one
Light source	LED — cheap	laser — expensive
Distance	> 100 m, but shorter than SMF	longest reach (km)
Cost	cheaper	pricier

Fiber speed names: **1000BASE-LX** (550 m MMF / 5 km SMF) · **10GBASE-SR** 400 m · **10GBASE-LR** 10 km · **10GBASE-ER** 30 km. Pattern: SR · LR · ER = longer and longer.

EXAM TRAPS !

- UTP's hard limit is **100 m** — 150 m between buildings = pick **MMF** ; 3 km apart = **SMF** .
- Two OLD routers wired with straight-through fail — they need a **crossover** cable.
- Same-role devices (PC looks like router/firewall electrically) always need crossover.

MUST REMEMBER

- PC / Router / Firewall: Tx on 1&2, Rx on 3&6 · switch is reversed
- Crossover for same type, straight-through for different type; MDI-X fixes it for you
- UTP copper maxes at **100 m** · and **1 byte = 8 bits**

Lesson 3 — The TCP/IP Model (& OSI) ☆

- **Protocol** = set of rules for communicating (a shared language). **Standard** = the vendor-neutral spec — why a Mac can talk to a Linux server.
- **IEEE** · LAN tech: Ethernet **802.3**, Wi-Fi **802.11** · **IETF** · internet protocols: TCP, IP, UDP, HTTP, DNS (published as **RFCs**).
- History flash: ARPANET 1969 (ran NCP) · TCP 1974 (Cerf & Kahn) · split into TCP + IP · **1983** TCP/IP adopted.

The 5 layers — the whole course hangs on this

L	Layer	Job	Key examples	PDU
5	Application	format & interpret process messages	HTTP, FTP, TFTP	data
4	Transport	process-to-process via port numbers (web 80, file 21)	TCP, UDP	TCP = segment · UDP = datagram
3	Internet	host-to-host across networks via IP addresses	IPv4, IPv6, ICMP — routers	packet
2	Local network	hop-to-hop delivery via MAC addresses	Ethernet, Wi-Fi — switches	frame
1	Physical	bits become signals (volt / light / radio)	UTP, fiber, NICs, antennas	bits

- **Hop** = one step from a **router/host** to the next router/host. **Switches do NOT count as hops** — they only extend the LAN.
- Transport “port” = just a **number** naming a process (web = 80) — nothing to do with RJ-45 ports!

Encapsulation, PDUs & payload

- **Sending (encapsulation)**: each layer adds its own **header** going down; **Layer 2 adds header AND trailer** (trailer = FCS error check). Receiving = exact reverse (decapsulation).
- PDU names: L4 TCP = **segment**, UDP = **datagram** · L3 = **packet** · L2 = **frame**.
- Only **frames** really travel on the wire — packets/segments always ride inside one.
- **Payload** = everything inside a PDU, NOT counting that layer’s own header/trailer.
- **Adjacent-layer interaction** = layers serving each other within one device. **Same-layer interaction** = matching layers “talking” device-to-device (IP to IP, MAC to MAC, port to port).

OSI 7 layers — lost the war, still on the exam

- ISO’s rival model: too late + too complex, so **TCP/IP won the real world** — but OSI survives as the reference/teaching model.

- Top · bottom: **Application, Presentation, Session, Transport, Network, Data Link, Physical** — mnemonic: “All People Seem To Need Data Processing”.
- Naming map: our **Internet (L3)** = Network layer · our **Local network (L2)** = Data Link layer · Application is often called **Layer 7**.

EXAM TRAPS !

- TCP makes **segments** , UDP makes **datagrams** — they love to ask this.
- Switches = not a hop; L2 trailer = only layer with one.
- A port number identifies a **process** , never a physical socket.

MUST REMEMBER

- L4 = ports/processes · L3 = IP + routers · L2 = MAC + switches · L1 = signals
- PDU ladder: segment/datagram · packet · frame
- OSI order: A-P-S-T-N-D-P

Lesson 4 — Cisco IOS CLI Basics ☆

IOS = Cisco's operating system (NOT Apple's iOS!). Network engineers live in the **CLI** (typed commands), not the GUI.

First-time connection (console)

- Laptop · device's **console port** (RJ-45 or USB Mini-B) using a **rollover cable** (pins reversed end-to-end: 1-8, 2-7 ... — it is **NOT** a crossover cable!) + USB-to-serial adapter.
- Software: a terminal emulator (e.g. **PuTTY**) · connection type **Serial** · Open.

MEMORIZE — SERIAL DEFAULTS

- **9600** baud · **8** data bits · **1** stop bit · **no** parity · **no** flow control

The mode ladder

```
Router>      # User EXEC: look only, default landing mode
Router#      # Privileged EXEC: full view, save, reboot (type: enable)
Router(config)# # Global config: make changes (type: configure terminal)
```

- Move up with **enable** then **configure terminal** (or **conf t**); step back with **exit**.
- Default hostname = **Router**.

Survival shortcuts

- **?** alone = list commands · **pass?** = completions of a word · word + space + **?** = what comes next (<cr> = just press Enter).
- **Tab** = autocomplete. Abbreviations work if unique: **en** = enable; too short (e.g. just **e**) = the **ambiguous** error.
- **do <cmd>** = run a Privileged EXEC show command from inside config mode (do show run).
- **no <cmd>** = undo / remove a command.

running-config vs startup-config

File	What it is	Danger
running-config	the LIVE config; every command edits it instantly	lost on reboot if never saved
startup-config	the config loaded at boot	only changes when YOU save

Router# write # 3 identical ways to save
 Router# write memory
 Router# copy running-config startup-config

Password ladder — always climb to the top

Level	Command	Stored as	Verdict
1	enable password CCNA	plain text in show run	never use
2	service password-encryption	Type 7 — weak, cracked online	meh
3	enable secret Cisco	Type 5 (MDS)	always use

- If BOTH password and secret exist · **only the secret is used** .
- service password-encryption does NOT touch the secret (always encrypted), never decrypts old passwords; turning it off only makes FUTURE passwords plain.
- Passwords are **case-sensitive** ; nothing shows on screen while typing; 3 wrong tries = “bad secrets”.

EXAM TRAPS !

- Rollover cable ≠ crossover cable.
- Secret beats password — exam asks which is used.
- Encrypting a switch? Remember: secret stays Type 5 regardless of the service.

MUST REMEMBER

- Modes: > user · # privileged · (config)# global config
- Save with write / copy run start — unsaved work dies at reboot
- Serial: 9600 / 8 / 1 / none / none · enable secret = best

Lesson 5 — Ethernet LAN Switching (I) ☆

- Switches operate at Layer 2. A LAN's edge is a **router interface**, never a switch — adding switches only extends the same LAN.
- Ethernet = Layer 1 (cables/signals) + Layer 2 (frames/MAC); this lesson = the Layer 2 half.

The Ethernet frame — 26 bytes of overhead

Field	Size	Job
Preamble	7 B	alternating 10101010 · syncs the receiver's clock
SFD	1 B	10101011 = "real frame starts NOW"
Destination MAC	6 B	where the frame is going
Source MAC	6 B	who sent it
Type / Length	2 B	≤ 1500 = length · ≥ 1536 = type: 0x0800 IPv4 · 0x86DD IPv6
FCS (trailer)	4 B	CRC check to detect corrupted frames

MAC addresses

- **48 bits = 6 bytes = 12 hex digits** · physical, burned in at the factory (**BIA** — burned-in address), globally unique.
- First 3 bytes = **OUI** (who made it), last 3 = the unique device · e.g. in **E8BA.7011.2874**, the OUI is **E8BA.70**.
- Hex 101: digits 0-9 then A-F = 10-15; columns count $\times 16$ (hex 10 = 16 decimal).

HOW A SWITCH THINKS — the core rule of switching

LEARN from the **SOURCE MAC**: record "this MAC lives out this port" in the **MAC address table**.

FORWARD using the **DESTINATION MAC**.

Frame type	Dest MAC in table?	Switch action
Known unicast	yes	forward out the ONE mapped port
Unknown unicast	no	FLOOD : every port except the one it arrived on

- Other PCs get the flooded copy, see it isn't theirs, and **drop** it; the real receiver replies, teaching the switch its MAC too — then no more flooding for that pair.
- Multi-switch: each switch learns **on its own** (no table sharing); a table entry means "direction toward that MAC", not necessarily a directly-plugged device.
- Dynamic entries expire after **5 minutes idle** (aging) — relearned on the next frame.

EXAM TRAPS !

- Clock sync = **Preamble** (not the SFD, not the FCS).
- OUI = first THREE bytes — E8BA.70, not E8BA, not E8BA.7011.
- Learning uses the SOURCE MAC; forwarding uses the DESTINATION MAC — most-asked question
- of the lesson.

MUST REMEMBER

- Frame overhead = 26 B · MAC = 48 bits · FCS uses CRC
- Unknown destination · flood; known destination · one port
- Dynamic MACs age out at 5 minutes

Lesson 6 — Switching (2): ARP, Ping, MAC Table ☆

Frame-size rules (the strict version)

- “True” Ethernet header = Dest + Src + Type; + FCS trailer · **18 bytes** (Preamble + SFD are NOT counted in the header).
- **Min frame 64 B** · **min payload 46 B** · too-small payloads are stuffed with **zero padding** (34 B ping · +12 B of 0s).

ARP: you know the IP, go find the MAC

Hosts are told to send to an **IP address**, but switches only speak **MAC**. **ARP bridges the gap** with two messages:

Message	Sent as	Plain-English meaning
ARP Request	broadcast FFFF.FFFF.FFFF	“Who has 192.168.1.3? Tell 192.168.1.1”
ARP Reply	unicast	“That’s me — here’s my MAC”

- The reply can be unicast because the request already carried the asker’s MAC.
- Switches flood the request like an unknown unicast; non-matching PCs drop it; the owner answers.
- Answers are cached in the **ARP table** (the IP-to-MAC map) so you only ask once.
- Commands: **arp -a** (Windows/macOS/Linux) · **show arp** (Cisco IOS).

Ping = ICMP reachability test

- Two messages: **ICMP Echo Request + Echo Reply** — both **always unicast**.
- Unicast needs the MAC first · **ARP runs before the first ping** · that’s why the first ping usually fails (the classic . ! ! ! !).
- IOS defaults: **5 echoes × 100 bytes** · . = fail · ! = success.
- In Wireshark you literally see: ARP req · ARP reply · ICMP req/reply, and EtherType **0x0806 = ARP**.

The switch’s MAC address table

```
SW1# show mac address-table
columns: VLAN | MAC Address | Type | Ports
```

- **VLAN** = virtual LAN (default 1, later lesson) · **Type** = dynamic · **Ports** = the interface to reach that MAC.
- Aging clears entries after **5 min** idle, or clear them by hand:

```
SW1# clear mac address-table dynamic          # everything
SW1# clear mac address-table dynamic address <mac>  # one MAC only
SW1# clear mac address-table dynamic interface <int> # one port only
```

Awareness: **Packet Tracer** = simulator (free, enough for CCNA); **GNS3** = runs real Cisco IOS images.

EXAM TRAPS !

- Flooded = **broadcast + unknown unicast** ONLY — known unicasts are never flooded.
- Don't mix tables: ARP table = IP/MAC/Type · MAC table = **VLAN/MAC/Type/Ports** .
- Exact hyphenation: **clear mac address-table** (older IOS wrote mac-address-table).

MUST REMEMBER

- ARP Request = broadcast, ARP Reply = unicast
- First ping fails — ARP hadn't finished yet
- Numbers: 18 B · 64 B · 46 B · padding = zeros

Lesson 7 — IPv4 Addressing ☆

Layer 3 = **routers**, **logical** addressing, and finding paths between **different** networks. An IPv4 address = **32 bits** = **4 octets** written in dotted decimal (each octet 0-255).

Reading binary (the skill everything below needs)

- Place values in one octet: **128 64 32 16 8 4 2 1** — add the values where the bit is 1.
- Ex: 10011111 = 128+16+8+4+2+1 = 159 · 01101110 = 64+32+8+4+2 = 110.
- Decimal · binary: greedily subtract 128, 64, 32 ... writing 1 when it fits · Ex: 221 = 11011101.
- Range per octet = **0-255** (255 = all ones).

Network portion vs host portion

- n** = the first n bits name the **network** (same for every device on it); the rest names the **host** (unique per device).
- /24** · first 3 octets = network (192.168.1.x) · **/16** · first 2 · **/8** · first 1.
- Devices with the same network portion are on the same LAN; different portion = a router must carry the traffic.

Address classes — decided by the FIRST octet

Class	1st octet	Starts with	Default prefix	Mask
A	0-127 (usable 1-126)	0	/8	255.0.0.0
B	128-191	10	/16	255.255.0.0
C	192-223	110	/24	255.255.255.0
D	224-239	1110	- multicast	—
E	240-255	1111	- experimental	—

- Trade-off: **A** = few networks but millions of hosts each · **C** = millions of tiny networks (256 addresses each) · **B** = the middle.
- 127.x.x.x** = **loopback** — ping yourself (traffic never leaves the PC, 0 ms), so real Class A starts at 1.
- Subnet mask = network bits as 1s, host bits as 0s — /24 and 255.255.255.0 are the **same fact** written two ways.

Network address & broadcast address

- Network address** = **host bits all 0** — names the network itself, can't be assigned (FIRST address of the range).

- **Broadcast address = host bits all 1** — reaches every host on the LAN, can't be assigned (LAST address).
- A Layer-3 broadcast (e.g. 192.168.1.255) maps to the Layer-2 broadcast MAC **FFFF.FFFF.FFFF** · and **routers NEVER forward broadcasts across** — that's why we split networks at all.

MUST REMEMBER

- 32 bits = 4 octets · place values 128/64/32/16/8/4/2/1
- Classes A/B/C = 1-126, 128-191, 192-223 with /8, /16, /24
- All-0s host = network · all-1s host = broadcast · neither is assignable
- Loopback = 127.x · broadcasts stop at the router

Lesson 8 — IPv4 (2) + Configuring Cisco Routers ☆

The money formula

Usable hosts = $2^n - 2$ (n = host bits; minus the **network** and **broadcast** addresses).

Network	Host bits	Usable hosts	First usable	Last usable	Broadcast
10.0.0.0/8	24	16,777,214	10.0.0.1	10.255.255.254	10.255.255.255
172.16.0.0/16	16	65,534	172.16.0.1	172.16.255.254	172.16.255.255
192.168.1.0/24	8	254	192.168.1.1	192.168.1.254	192.168.1.255

- Rules: **first usable** = **network** + 1 · **last usable** = **broadcast** - 1. Only host-portion octets ever change.
- Common convention: PCs take the **first usable**, the router interface takes the **last usable** (a habit, not a law).

Interface status reality check

- Router interfaces come **administratively down** by default (the shutdown command is pre-applied) — YOU must **no shutdown** them.
- Switch interfaces are **NOT** shut down — they come up as soon as a cable clicks in.
- In show output: **Status** = **Layer 1** (physical), **Protocol** = **Layer 2** (data link). L1 down · L2 can NEVER be up · but L1 up + L2 down happens.

Giving RI an IP address — full walkthrough

```
Router> enable
Router# configure terminal
Router(config)# interface g0/0
Router(config-if)# ip address 10.255.255.254 255.0.0.0
Router(config-if)# no shutdown
Router(config-if)# int g0/1          # jump straight between interfaces
Router(config-if)# ip address 172.16.255.254 255.255.0.0
Router(config-if)# no shut
Router(config-if)# do show ip interface brief  # verify without exiting
```

- Cisco CLI wants the mask in **dotted decimal (255.0.0.0)** — NOT /8!
- After no shutdown you see %LINK (L1) and %LINEPROTO (L2) come up; the brief shows **Method** = **manual**, up/up.

Verification & documentation commands

- **show ip interface brief** = the quickest health check (IP, method, status, protocol).
- **show interfaces g0/0** = deep detail: line status (L1), line protocol (L2), MAC + BIA (factory MAC — it stays even if you override the MAC), Internet address/prefix.
- **description <text>** (shortcut **desc**) labels an interface; view all with **show interfaces description** .

EXAM TRAPS !

- Forgetting **no shutdown** on a router — the #1 lab mistake.
- Typing /24 instead of 255.255.255.0 — CLI rejects slash masks.
- **down/down** (nothing cabled) vs **administratively down** (shutdown applied) — read carefully!

MUST REMEMBER

- $2^n - 2$ usable hosts · first = network+1 · last = broadcast-1
- Router ports default DOWN; syntax: ip address <ip> <dotted mask>
- Status = L1, Protocol = L2 · BIA = original factory MAC

Lesson 9 — Switch Interfaces: Speed & Duplex ☆

Reading switch show output

Command	Shows	Status words
show ip interface brief	IP + L1/L2 per port	up/down/administratively down
show interfaces status	port, desc, status, VLAN, duplex, speed, type	connected / notconnect / disabled

- Switch ports show IP “unassigned” — normal! Switching is Layer-2 work; ports don’t need IPs.
- The **Name** column in show interfaces status is actually your **description** text.

Duplex, and the hub story

- **Full duplex** = send + receive at the same time (**modern LANs are always full**). **Half duplex** = one at a time.
- **Hub** = dumb **Layer-1 repeater** : floods every frame out every port · all ports share **ONE collision domain** · forced into half duplex.
- **Switch** = Layer 2, forwards by MAC · **each port = its own collision domain** · full duplex becomes safe.
- **CSMA/CD** (half duplex only): listen for silence · send · if collision: **jam signal** · wait a **random** time · retry.

Autonegotiation — and the classic mismatch

- Defaults: **speed auto + duplex auto** ; two autonegotiating devices agree on the fastest common speed + full duplex.
- If the NEIGHBOR disables autonegotiation: the switch can **sense SPEED** (if it can’t, it falls back to the slowest — 10 Mbps) but it **CANNOT sense duplex** .
- Cisco’s rule: **speed ≤100 Mbps** · assume **HALF** duplex · **speed ≥1000 Mbps** · assume **FULL** duplex.
- Classic exam case: neighbor fixed at 100/full · switch picks **100/ half** · **duplex mismatch** = collisions + terrible performance (full side never waits, half side expects politeness).
- **Golden rule: run autonegotiation on BOTH ends.**

Manual config + interface ranges

```
SW1(config)# interface f0/1
SW1(config-if)# speed 100
SW1(config-if)# duplex full
SW1(config)# interface range f0/5 - 6, f0/9 - 12  # pick several ports at once
SW1(config-if-range)# description Unused
SW1(config-if-range)# shutdown                # secure the unused ports
```

- In show interfaces status, **a-100 / a-full** means the setting was **auto-negotiated** ; no "a-" prefix = manually forced.
- **Shut down unused ports** — an open port is free network access for anyone who walks in.
- Ranges can skip ports with commas, and the prompt becomes **(config-if-range)#** .

Error counters (show interfaces <name>)

Counter	Counts
runs	frames smaller than 64 B
giants	frames larger than 1518 B
CRC	frames that FAILED the FCS/CRC check
frame	frames with an illegal format
input / output errors	totals of failed receives / failed sends

Same command and counters work on routers as well as switches.

EXAM TRAPS !

- Mismatch math: neighbor 100/full, autoneg off · answer is **100 Mbps, HALF duplex** .
- **notconnect** (no cable) vs **disabled** (shutdown applied) vs connected.
- CSMA/CD belongs to hubs / half duplex — modern full-duplex switch ports don't need it.

MUST REMEMBER

- Each switch port = ONE collision domain · full duplex
- ≤100-half, ≥1000-full when autoneg can't be used
- runs <64 B, giants >1518 B, CRC = FCS failures

Lesson 10 — The IPv4 Header ☆

The **packet** (Layer-3 PDU) = IPv4 header + data. Header size: min **20 bytes**, max **60 bytes**. Know what each field solves :

Field	Bits	What it's for
Version	4	always 0100 (4) for IPv4 (IPv6 = 0110)
IHL	4	header length in 4-byte steps : 5 = 20 B (min) · 15 = 60 B (max)
DSCP	6	QoS — prioritize delay-sensitive traffic (voice/video)
ECN	2	signal congestion WITHOUT dropping packets (needs both ends + path support)
Total Length	16	whole packet size in bytes · max 65,535 B
Identification	16	groups fragments that belong to one original packet
Flags	3	bit0 reserved · DF don't fragment · MF more fragments
Fragment Offset	13	where this piece sits · reassembles out-of-order arrivals
TTL	8	minus 1 per router ; at 0 the packet is dropped — kills routing loops (default 64)
Protocol	8	what's inside: 1 ICMP · 6 TCP · 17 UDP · 89 OSPF
Header Checksum	16	error-check of the header ONLY
Source IP / Dest IP	32 + 32	sender and receiver addresses (Lessons 7–8)
Options	0–320	rarely used; present exactly when IHL > 5

Fragmentation — the fields team up

- **MTU = largest packet a link can carry, usually 1500 B.** Too big · the packet is sliced into fragments, each with its own full header.
- Reassembly keys: same **Identification** = same family · **MF = 1** on every fragment except the LAST · **Fragment Offset** = jigsaw position so order doesn't matter.
- **DF = 1 means NEVER fragment** — if it can't fit whole, delivery simply fails (useful for discovering MTU problems).
- Wireshark proof: a 10,000-byte ping arrives as several 1500-B packets sharing one ID, MF=1 on all but the last.

Who checks errors where (layered responsibility)

- **IPv4 Header Checksum** · header only — a mismatch · router drops the packet.
- The **data inside** is checked by Layer 4 (TCP/UDP have their own checksums).
- The **frame** is checked hop-by-hop by the Layer-2 FCS/CRC (Lesson 5).

EXAM TRAPS !

- TTL reaching **0** — dropped by the ROUTER (prevents infinite loops).
- Version field is fixed **0100** ; don't confuse Total Length (whole packet, bytes) with IHL (header only, $\times 4$ -byte units).
- Header checksum never looks at the data — that's TCP/UDP's job.

MUST REMEMBER

- Header = $20 - 60 \text{ B} \cdot \text{IHL} \times 4 = \text{header length}$
- Protocol numbers: 1 / 6 / 17 / 89 · TTL default 64
- Fragment trio: Identification + MF bit + Fragment Offset · DF blocks it all

Final Cram Card — every number the exam loves ☆

Numbers & formulas

Fact	Value
Octet place values	128 64 32 16 8 4 2 1 (octet range 0-255)
1 byte	8 bits · speed in bits, storage in bytes
Usable hosts	$2^n - 2$ · first usable = network+1 · last usable = broadcast-1
Classes	A 1-126 /8 · B 128-191 /16 · C 192-223 /24 (127.x = loopback, D = multicast, E = experimental)
Masks	/8 = 255.0.0.0 · /16 = 255.255.0.0 · /24 = 255.255.255.0
EtherTypes	0x0800 IPv4 · 0x86DD IPv6 · 0x0806 ARP
IP Protocol field	1 ICMP · 6 TCP · 17 UDP · 89 OSPF
Well-known ports	21 FTP (file) · 80 HTTP (web)
Byte budgets	frame overhead 26 B (18 strict) · min frame 64 B · min payload 46 B · IPv4 header 20-60 B · MTU 1500 B
Timers & counters	TTL default 64 · MAC aging 5 min · IOS ping = 5 × 100 B
Cables	UTP max 100 m · crossover for same-type devices · MDI-X ignores cable type · serial 9600/8/1/none/none
CLI ladder	> user · # privileged · (config)# global · save = copy run start · secret beats password

60-second self-test (say the answers out loud)

- 1) Two routers, one old straight-through cable — works or not? Ans: no — need crossover / MDI-X
- 2) Switch gets an unknown unicast — what does it do? Ans: floods all ports except the source port
- 3) 192.168.1.0/24: network, broadcast, usable hosts? Ans: .0 · .255 · 254
- 4) ARP request: broadcast or unicast? Reply? Ans: broadcast · unicast
- 5) TTL hits 0 — then what? Ans: the router drops it
- 6) Convert 201 to binary. Ans: 11001001
- 7) Neighbor fixed 100/full, autoneg off — your switch's settings? Ans: 100 Mbps, HALF
- 8) Which PDU at L3? L2? TCP L4? UDP L4? Ans: packet · frame · segment · datagram